

Single Sign-On, Three Ways

A walkthrough of three runnable OIDC learning projects — from authentication inside the app, to the gateway, to a dedicated forward-auth service.

Demo 1 **sso-demo** — OIDC *in the app* (each backend is its own client)

Demo 2 **sso-demo2** — OIDC *at the gateway* (Kong plugin + Redis)

Demo 3 **sso-demo-3** — OIDC in a *forward-auth service* (Traefik + auth-service)

All three use the same identity foundation: Keycloak as an OpenID Provider and identity broker in front of Microsoft Entra ID, realm users persisted to Postgres, in-house users signing in through Entra (role: **employee**) and outside users self-registering with email + password (role: **customer**).

The common foundation

Every demo solves the same problem — let in-house staff sign in with Microsoft and let outside users register with email + password — and shares the same identity layer. What changes between them is **where authentication and authorization happen**.

The building blocks (identical across all three)

Component	Role
Microsoft Entra ID	The real identity provider for in-house users. Lives in Microsoft's cloud; set up by hand once (an app registration whose redirect URI is Keycloak's broker endpoint).
Keycloak	The OpenID Provider the apps trust, <i>and</i> an identity broker that delegates in-house logins to Entra. Also hosts the self-registration page for outside users. Swapping Entra for Google/Okta later needs zero app changes — that is the point of brokering.
Postgres	Keycloak's user store. Every account — brokered from Entra <i>and</i> self-registered — is persisted here. (Demo 1 used Keycloak's in-memory DB; Demos 2 & 3 wire in Postgres.)
Roles	A hardcoded mapper grants <code>employee</code> to anyone who signs in through Entra; self-registered users get <code>customer</code> . Apps use these for access decisions.
Redis	Shared session store (Demos 2 & 3). Keeps sessions out of any single node's memory so the auth layer can scale horizontally.

The two-app single sign-on test

Each demo runs **two independent apps**. Sign in to App One, then open App Two: it completes login with **no prompt**, because the browser already holds a Keycloak SSO session. That silent second login is single sign-on. The apps share no cookies or sessions — only the same Keycloak realm.

The recurring pitfall — issuer / hostname. OIDC tokens embed the issuer URL (`iss`), and the client must validate it. The browser reaches Keycloak at `localhost:8080` , but a container reaches it at `keycloak:8080` . If the client discovers Keycloak under one name while tokens are minted under another, validation fails. Each demo fixes this differently (see below) — it is the single most common thing to get wrong.

The three architectures at a glance

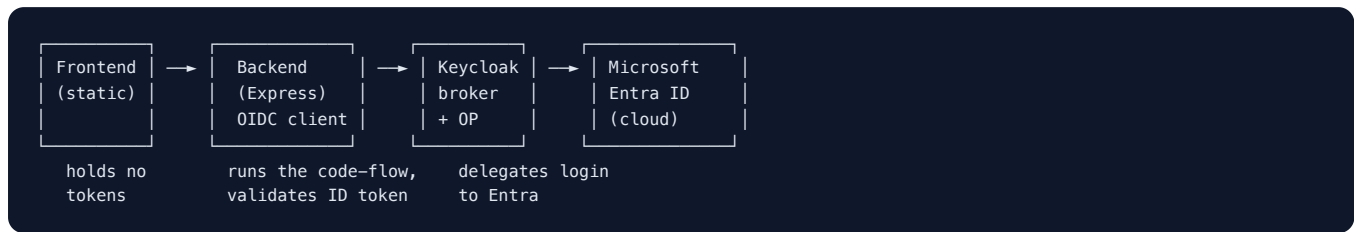
	Demo 1 sso-demo	Demo 2 sso-demo2	Demo 3 sso-demo-3
Who is the OIDC client?	Each backend (one per app)	Kong (the gateway), via the <code>kong-oidc</code> plugin	One auth-service
Authentication happens...	In every app	At the gateway	In the auth-service
Authorization (role checks)...	In every app	In every app (each backend checks roles)	In the auth-service (centralized policy)
Edge technology	None (direct to backend)	Kong API gateway	Traefik + ForwardAuth middleware
Sessions	In-memory (per backend)	Redis (Kong / lua-resty-session)	Redis (auth-service / express-session)
App code does auth?	Yes — all of it	Reads an injected header; still checks roles	No — zero auth code
Issuer/hostname fix	<code>customFetch</code> host rewrite	<code>localhost:host-gateway</code>	<code>customFetch</code> host rewrite

The progression. Each demo pushes auth further out of the application. Demo 1 duplicates security-critical code in every service. Demo 2 lifts *authentication* to the gateway but leaves *authorization* scattered. Demo 3 centralizes both, leaving the apps completely free of auth logic — at the cost of a new hot-path dependency.

Demo 1

sso-demo — OIDC inside the application

The classic, self-contained approach: each backend is a confidential OIDC client and runs the whole login flow itself.



How it works

- The browser hits the backend's `/login`; the backend builds an Authorization Code + PKCE request and redirects to Keycloak (with `kc_idp_hint=microsoft` to jump straight to Entra).
- After Entra authenticates the user, Keycloak issues its own code back to the backend's `/auth/callback`. The backend exchanges it server-to-server, and `openid-client` verifies the ID token's signature, issuer, audience, expiry and nonce.
- Claims go into an Express session (an `httpOnly` cookie). The frontend calls `/api/me` to render name + email. It never sees a token.
- Logout destroys the session *and* redirects to Keycloak's end-session endpoint (RP-initiated logout) so the SSO session dies too.

Issuer fix. The backend validates against `localhost:8080` but rewrites the host to `keycloak:8080` at the HTTP layer via `openid-client`'s `customFetch` hook — issuer checks see "localhost", packets go to "keycloak".

Pros

- Simple to understand; no extra infrastructure.
- Full control of the flow in app code.
- Great for learning OIDC mechanics end to end.

Cons

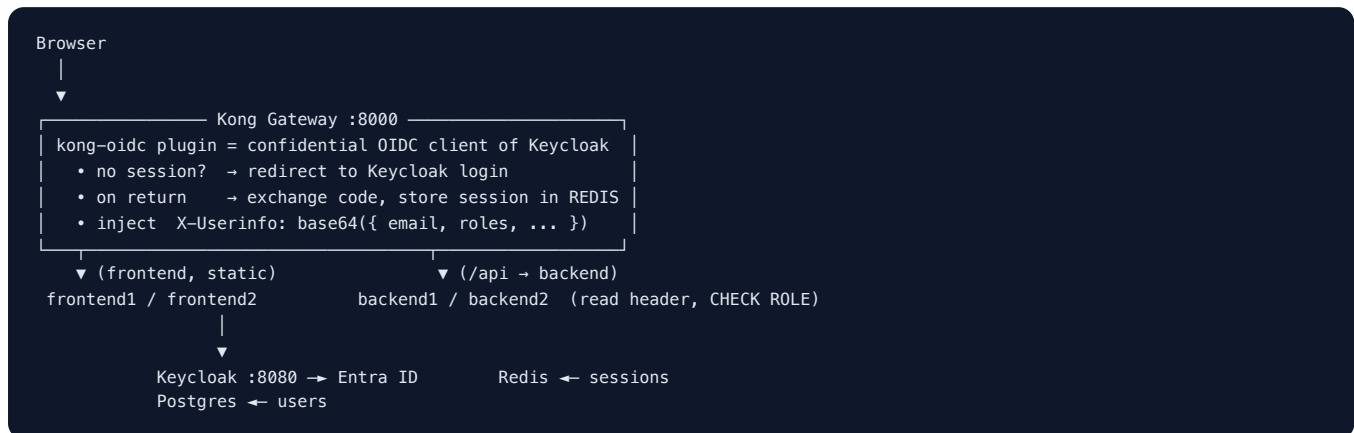
- Security-critical code **duplicated in every service** — easy to drift.
- Every team must re-implement auth correctly in their language.
- In-memory sessions don't survive restarts or scale across instances.

Use when: a single app, a prototype, or you are learning how OIDC actually works.

Demo 2

sso-demo2 — OIDC at the gateway (Kong)

Authentication moves out of the apps and up into an API gateway. Kong is the OIDC client; the apps behind it read a header Kong injects.



What's new vs Demo 1

- **Kong** runs the OIDC flow once, at the edge. The apps contain no `openid-client`, no token handling — they just decode the `X-Userinfo` header Kong attaches.
- **Redis sessions** ("sticky"/shared): only a session id rides in the cookie; the payload lives in Redis, so any Kong replica can resolve any session and the load balancer is free to route anywhere.
- **Postgres** backs Keycloak, so all users are durably stored.
- **Dual login** on one Keycloak page: a Microsoft button (in-house → `employee`) and a Register link (outside → `customer`).

Authentication vs authorization here

Kong answers "**are you logged in?**" (authentication). Each backend still answers "**do you have the right role?**" (authorization): App One (Customer Portal) lets any authenticated user in; App Two (Internal Tools) requires `employee` and returns `403` to a customer. So authZ is still scattered across the apps.

Issuer fix. Kong's Lua stack has no fetch-rewrite hook, so the container is given `extra_hosts: localhost:host-gateway` — its `localhost` resolves to the host machine where Keycloak's 8080 is published. One issuer string everywhere.

Pros

- Authentication implemented **once**, not per app.
- Apps shed all token/session code.
- Shared Redis sessions enable horizontal scaling.
- Language-agnostic: any backend just reads a header.

Cons

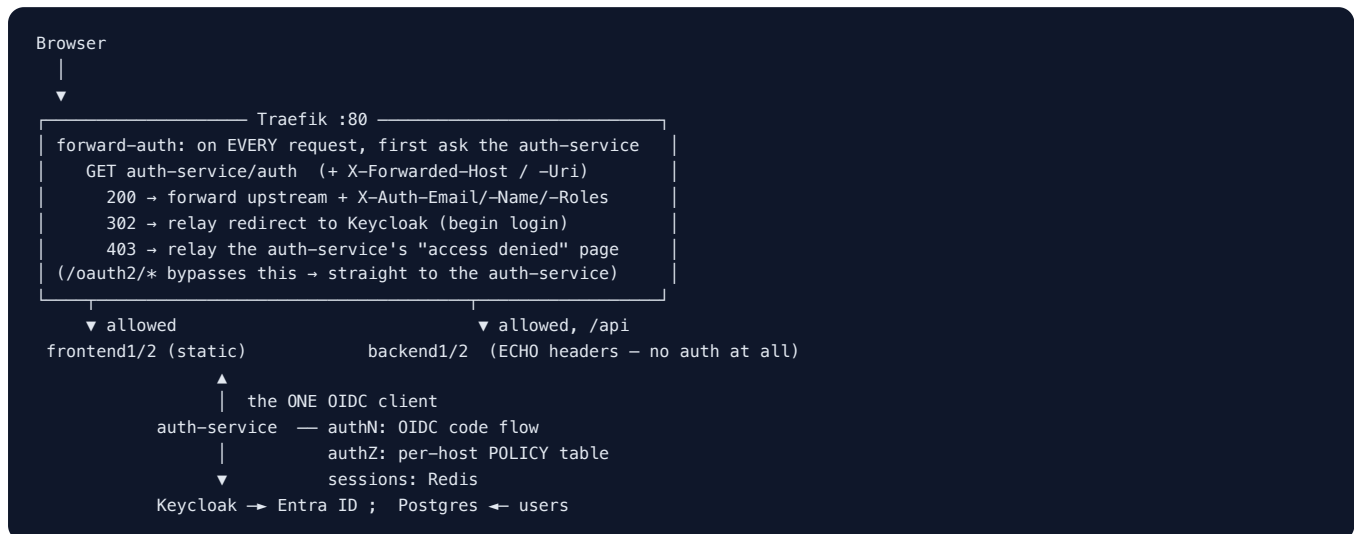
- Authorization still lives in every app (role checks duplicated).
- The `kong-oidc` community plugin is version-sensitive.
- The gateway is a new critical dependency to operate.

Use when: several apps should share one login, but each app owns its own (possibly resource-specific) authorization rules.

Demo 3

sso-demo-3 — forward-auth service (Traefik)

A single auth-service does **both** authentication and authorization at the edge. The apps become truly zero-auth — not even a role check.



The request lifecycle

- Traefik calls `auth-service/auth` before routing *any* request, passing the original `X-Forwarded-Host/-Uri`.
- **No session** → the auth-service returns `302` to Keycloak; Traefik relays it. After login, Keycloak hits `/oauth2/callback` (which bypasses forward-auth, or login would loop), the auth-service stores the user + roles, and redirects back.
- **Session + allowed** → `200` with `X-Auth-Email/-Name/-Roles`, which Traefik copies onto the upstream request. The backend just echoes them.
- **Session + wrong role** → `403` with a friendly deny page; the app is never reached.

authN vs authZ, in one place

Authentication is the OIDC flow + session lookup. Authorization is a single centralized table — change a rule once, it applies everywhere:

```
const POLICY = {
  'app-one.localhost': { requireRole: null }, // any authenticated user
  'app-two.localhost': { requireRole: 'employee' }, // employees only
};
```

A customer opening App Two authenticates fine, then the policy denies them — the app never runs. Swap this object for OPA/Rego or Cedar and nothing in the apps changes.

Pros

- **One place for all policy** — authN and authZ both centralized.
- Apps are truly zero-auth, in any language.
- Uniform logging and one audit trail for every access decision.
- Change a rule once; no app redeloys.

Cons

- A synchronous auth call on **every** request (latency).
- Single point of failure / bottleneck — must be HA + fast.
- Body-blind: object-level checks ("edit *this* doc") still need the app.
- Central policy is a coupling point — one bad deploy affects all apps.

Use when: many heterogeneous services need consistent, centrally governed access control. **Overkill when:** a single app, or rules need per-resource context the gateway can't see.

Choosing between them

If you...	Reach for
Are learning how OIDC actually works, or shipping one app	Demo 1 in-app OIDC
Have several apps that should share one login, but each owns its own (often resource-level) authorization	Demo 2 gateway authentication
Have many services and want authentication <i>and</i> coarse authorization governed centrally and consistently	Demo 3 forward-auth service

A common middle ground in practice

Real systems often blend these: the gateway/forward-auth layer handles authentication plus *coarse* authorization (route- and role-level gates), while individual apps keep only the *object-level* authorization that genuinely needs request context the edge can't see (e.g. "is Alice the owner of this specific record?"). You don't have to pick one model for everything.

What stays the same regardless of model

- Brokering through Keycloak means the apps never touch Microsoft directly — swapping or adding identity providers is a Keycloak change, not an app change.
- Tokens are validated, not trusted: signature, issuer, audience, expiry and nonce are always checked by whoever is the OIDC client.
- The issuer/hostname consistency rule is non-negotiable — the URL the browser uses and the URL the client validates against must agree.
- Sessions belong in a shared store (Redis) the moment you run more than one instance of whatever holds them.

All three are local-learning stacks. Plain HTTP, `secure=false` cookies, dev-mode Keycloak, and hardcoded secrets are deliberate conveniences. Production needs TLS everywhere, a secrets manager, HA for every shared component, and a hardened trust boundary so backends can't be reached (and their identity headers forged) except through the edge.

Each project ships a detailed README with setup, the full login flow, the two-app SSO test, and troubleshooting. This document summarizes and compares them.